GUIA DO PROFESSOR

Servomotores: do primeiro movimento ao braço robótico

Plano de aulas, fundamentação técnica, solução de problemas e avaliação

---

Servo: Tower Pro SG90 (micro servo 9g) • **Placas:** Arduino Uno/Nano → ESP32

Projeto final: Servo Mini Arm (LucaDilo, MakerWorld 1290483) em modo teach & repeat

Público: Ensino Médio / Técnico • **Carga horária:** 6 encontros de 100 min (ou 12 aulas de 50 min)

Espaço Cria — 2026

Sumário

1. Visão geral da sequência didática
2. Materiais
3. Preparação antes das aulas
4. Plano dos encontros
5. Fundamentação técnica para o professor
6. Erros comuns e como resolver
7. Respostas e comentários dos desafios
8. Avaliação
9. Diferenciação e inclusão
10. Segurança no laboratório
11. Referências

Anexo A — Sketches de referência

Anexo B — Folhas de calibração (cópias para distribuir)

1. Visão geral da sequência didática

Esta sequência leva o aluno de "o que é um servo" até um braço robótico que **grava e repete um ciclo de trabalho** de forma autônoma. A progressão é deliberadamente **incremental**: cada encontro reutiliza o código do anterior e acrescenta um único conceito novo. O aluno chega ao braço robótico já dominando as ferramentas (PWM, ADC, `map()`, Serial, interpolação), de modo que o projeto final é integração, não novidade.

Objetivos de aprendizagem

Ao final, o aluno deverá ser capaz de:

- Explicar como a largura de pulso controla a posição de um servomotor e por que a frequência é 50 Hz.
- Ler um sensor analógico (potenciômetro) e converter a leitura em ação física usando `map()`.
- **Calibrar** um atuador: medir limites reais e incorporá-los ao código.
- Migrar um projeto entre plataformas (Uno → ESP32), identificando o que muda (tensão, resolução do ADC, bibliotecas).
- Montar um mecanismo impresso em 3D respeitando pré-requisitos (centralização dos servos).
- Definir limites de segurança por software e explicar por que isso protege o hardware.
- Implementar e explicar o paradigma **teach & repeat** com persistência em memória não volátil.
- Depurar sistematicamente com o Monitor Serial e com uma tabela de sintomas → causas.

Competências e habilidades relacionadas (BNCC / Itinerários)

- **Pensamento computacional:** decomposição (juntas em vetores), abstração (`struct Pose`), algoritmos (interpolação), depuração.
- **Cultura digital / Mundo do trabalho:** automação industrial, robótica, prototipagem digital (impressão 3D).
- **Matemática (EM13MAT):** proporcionalidade (`map()`), funções lineares, médias, medidas e erro.
- **Ciências da Natureza (EM13CNT):** circuitos elétricos, divisor de tensão, corrente, potência e dissipação.
- **Trabalho em grupo, comunicação técnica** (folhas de calibração, apresentação final).

Pré-requisitos

- Ter piscado um LED e usado o Monitor Serial no Arduino (1 aula de nivelamento resolve).
- Noções de tensão/corrente e de protoboard.
- Nenhum pré-requisito de programação além de `if`, `for` e variáveis.

2. Materiais

Por grupo (recomendado: 3 alunos)

Item	Qtd	Custo aprox. (R$)	Observação
Arduino Uno ou Nano + cabo	1	40–90	Encontros 1–2 (pode ser substituído pelo ESP32 desde o início, ver §5)
ESP32 Dev Module (30/38 pinos) + cabo micro-USB	1	35–60	Encontros 3–6
Servo SG90	5	12–18 cada	4 para o braço + 1 reserva (quebra é comum na primeira montagem)
Potenciômetro 10 kΩ linear (com knob)	4	2–4 cada	
Protoboard 830 pontos + jumpers	1 kit	25–40	
Fonte 5 V 2–3 A (plugue P4) + adaptador P4-borne	1	20–35	Obrigatória para 4 servos
Capacitor eletrolítico 470–1000 µF / 10 V	1	1–2	Evita brownout do ESP32
Push-buttons + LED + resistor 220 Ω	2 + 1	3	
Peças impressas do Servo Mini Arm	1 conjunto	~15 (filamento)	≈ 80–120 g de PLA; ver §3
Parafusos M3×12 e M2×5	~8 + ~8	5	Ou o que couber: vários makers usam os do kit do servo
Cabos extensores de servo 20 cm	2–4	3 cada	
Objeto de teste (cubo 2–3 cm, tampinha)	2	—	

Para a turma

- Impressora 3D (ou serviço de impressão) — imprimir **com 2 semanas de antecedência**.
- Ferro de solda (opcional: apenas se for encurtar cabos de servo), alicate de corte, chave Phillips pequena, estilete para rebarbas.
- Multímetro (1 a cada 2 grupos): medir 5 V da fonte e continuidade do GND comum.
- Etiquetas para numerar servos e caixas para guardar o braço entre encontros.

3. Preparação antes das aulas

Software (computadores do laboratório)

1. Arduino IDE 2 com o core **esp32 by Espressif** (URL de placas: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`).

2. Bibliotecas: **ESP32Servo** (Kevin Harrington). `Servo`, `EEPROM` e `Preferences` já vêm com os cores.
3. Drivers USB-serial: **CP210x** (Silicon Labs) e **CH340** (WCH) — a maioria dos ESP32 baratos usa um dos dois. Instale nos PCs antes do Encontro 3.
4. Copie a pasta `aula-servos/codigo/` para cada máquina. Todos os sketches foram **compilados e verificados** (Uno: 01, 02, 03, 07; ESP32: 04, 05, 06).
5. Os esquemas de montagem deste guia foram feitos no **Fritzing** e estão em `aula-servos/esquemas/saida/*.fzz` (editáveis; Fritzing portátil em `C:FerramentasFritzing`) e `*.png` para projetar em sala.
6. Alternativa por linha de comando (Arduino CLI): `arduino-cli compile --fqbn esp32:esp32:esp32 05_braco_4pots_esp32` e `arduino-cli upload -p COM3 --fqbn esp32:esp32:esp32 05_braco_4pots_esp32`.

Impressão 3D

- Modelo: **Servo Mini Arm** — LucaDilo — makerworld.com/models/1290483. Use o perfil “Mini Braço Servo” (inclui as atualizações de 21/05/2025 com canal para cabos) e a “Base com Compartimento para Eletrônicos”.
- PLA, camada 0,2 mm, 3 perímetros, 15–20 % de preenchimento. Tempo total ≈ 6–9 h por conjunto (varia com a impressora). Verifique quais peças pedem suporte no perfil.
- Imprima **1 conjunto extra** de peças pequenas (dedos da garra, suportes de servo): são as que mais quebram.
- Baixe também o **Servo Mini Arm Assembly Sheet.pdf** e o código de referência do autor: `github.com/LucaDiLorenzo98/mini_servo_arm_code` (útil para comparar abordagens com os alunos).
- **Base:** o original usa 1 servo de rotação contínua 360°. Com o SG90 comum (180°) a base gira ±90°, o que atende à atividade. Se quiser o original, use um FS90R ou um SG90 modificado.

Hardware

1. Teste **todos os servos** com o sketch 01 antes da aula. Separe os defeituosos.
2. Confira a fonte 5 V com multímetro; fontes baratas entregam 4,3 V com carga.
3. Monte um braço-modelo você mesmo antes. Anote o tempo (≈ 1,5–2 h na primeira vez) e os pontos difíceis para orientar os grupos.
4. Deixe um ESP32 já gravado com o sketch 06 e poses salvas: é a demonstração de abertura do curso.

4. Plano dos encontros

Cada encontro tem 100 min. Os tempos são sugestões; a coluna **Evidência** indica o que observar/recolher para avaliação formativa.

Encontro 1 — O que é um servo e o primeiro movimento

Tempo	Atividade	Professor	Alunos	Evidência
10'	Abertura	Demonstra o braço-modelo executando um ciclo. Pergunta: “como ele sabe onde parar?”	Levantam hipóteses	Hipóteses no quadro
20'	Teoria: servo e pulso	Explica o interior do servo e a tabela pulso × ângulo (Guia §2). Mostra o pulso no osciloscópio ou no plotter serial	Anotam a tabela	—
15'	Ligações	Circula conferindo cor dos fios e polaridade	Montam servo + Uno	Checklist (Anexo 3)
25'	Sketch 01 + Experimente	Orienta a alterar delays e limites do `for`	Executam e registram observações	Respostas do “Experimente”
25'	Desafios 1.1–1.3	Atende grupos; avança os rápidos para 1.3	Programam	Sketch funcionando
5'	Fechamento	“O que faz o movimento ser suave?”	Respondem	Saída rápida (ticket)

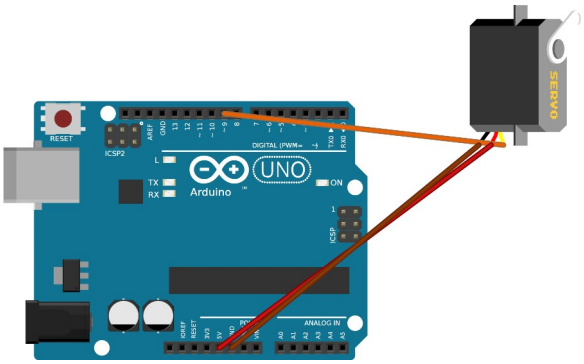


Figura 1 — Montagem do Encontro 1 (sketch 01). Conferir a ordem marrom/vermelho/laranja em cada grupo.

Encontro 2 — Potenciômetro e calibração

Tempo	Atividade	Professor	Alunos	Evidência
15'	Divisor de tensão e ADC	Mede com multímetro o cursor do pot em 3 posições; relaciona com 0..1023	Preveem valores	Previsões × medidas
20'	Sketch 02	Enfatiza o Monitor Serial como ferramenta de depuração	Montam e testam	—
15'	Experimente 1–4	Provoca: “e se trocar 0..1023 por	Modificam o `map()`	Explicação oral

Tempo	Atividade	Professor	Alunos	Evidência
		1023..0?”		
30'	Sketch 03 + Folha A	Mostra o que é o batente e o ronco do servo forçando. Exige o preenchimento da folha para **cada** servo	Calibram e etiquetam servos	**Folha de calibração A**
15'	Desafio 2.1 ou 2.3	Zona morta / movimento suave (2.3 é essencial para o braço)	Programam	Sketch
5'	Fechamento	“Por que dois servos ‘iguais’ deram pulsos diferentes?”	Discutem	—

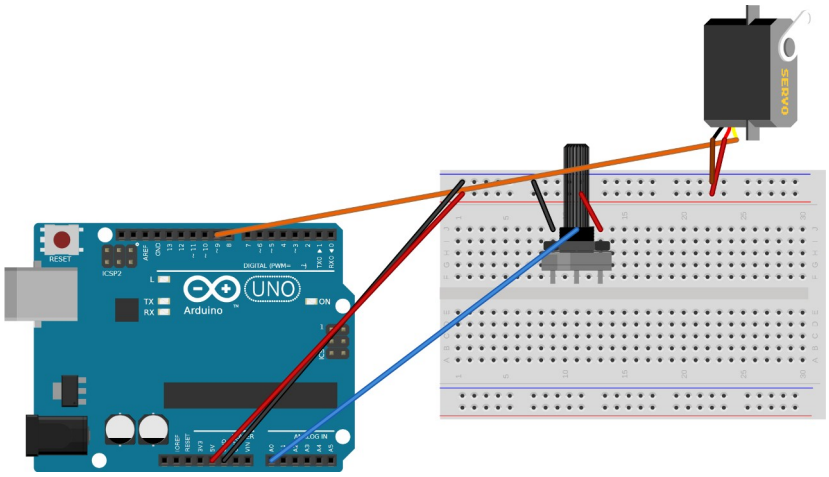


Figura 2 — Montagem dos Encontros 2 (sketches 02 e 03). Erro mais comum: cursor do pot fora de A0 ou pot ligado só em uma ponta.

Encontro 3 — Migrando para o ESP32

Tempo	Atividade	Professor	Alunos	Evidência
15'	Por que ESP32?	Compara as placas (tabela do Guia §7). Insiste em 3,3 V no pot	—	—
15'	Configurar a IDE	Instala core/lib nos PCs que faltarem; drivers	Configuram	Porta COM visível
25'	Sketch 04	Se “Failed to connect”: segurar BOOT	Montam e gravam	Servo seguindo o pot
25'	Desafios 4.1–4.2	4.1 é a calibração refeita no ESP32; comparar com a Folha A	Recalibram	Folha A (coluna ESP32)
15'	Não linearidade do ADC	Plotter serial: leitura do pot × posição do knob. Discutir por que as pontas “achatam”	Observam	Gráfico
5'	Fechamento	Lista o que mudou Uno→ESP32	Completam a lista	Lista

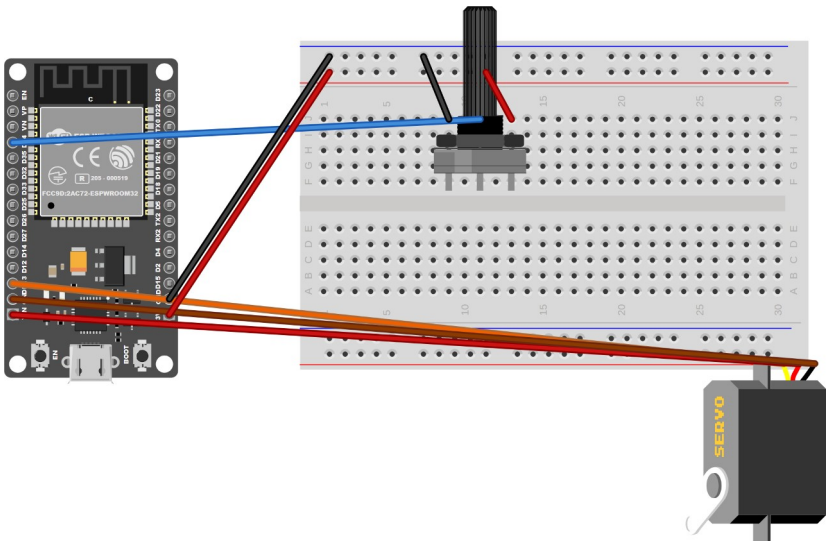


Figura 3 — Montagem do Encontro 3 (sketch 04). Ponto de atenção: pot em 3V3, servo em VIN.

Encontro 4 — Montagem do Servo Mini Arm

Tempo	Atividade	Professor	Alunos	Evidência
10'	Regra de ouro	**Centralizar servos antes de montar.** Demonstra com o sketch 01 fixo em `write(90)`	—	—
20'	Centralização	Confere cada servo: energizado → centro → desliga → horn	Centralizam os 4 servos	Marca de caneta no eixo
60'	Montagem	Circula com o Assembly Sheet; ajuda nos cabos e nos parafusos M3 (não apertar demais)	Montam	Braço montado
10'	Teste junta a junta	Sketch 02 em cada servo, um por vez, com fonte externa	Testam	Cada junta move sem bater

Encontro 5 — Marionete e limites mecânicos

Tempo	Atividade	Professor	Alunos	Evidência
15'	Alimentação de 4 servos	Mede a corrente com o multímetro em série: parado × movendo × garra forçando. Mostra o ESP32 reiniciando sem capacitor	Anotam correntes	Tabela de correntes
20'	Ligações completas	Confere GND comum com multímetro	Montam	Checklist
20'	Sketch 05	Explica vetores e `allocateTimer`	Gravam e movem o braço	Braço respondendo aos 4 pots
30'	Limites + Folha B	Ensina a “sentir” o encosto e recuar 5°. Garra: encostar, não	Definem `ANG_MIN/MAX`	**Folha de calibração B**

Tempo	Atividade	Professor	Alunos	Evidência
		forçar		
15'	Desafio 5.1	Pose de descanso por botão (prepara o Encontro 6)	Programam	Sketch

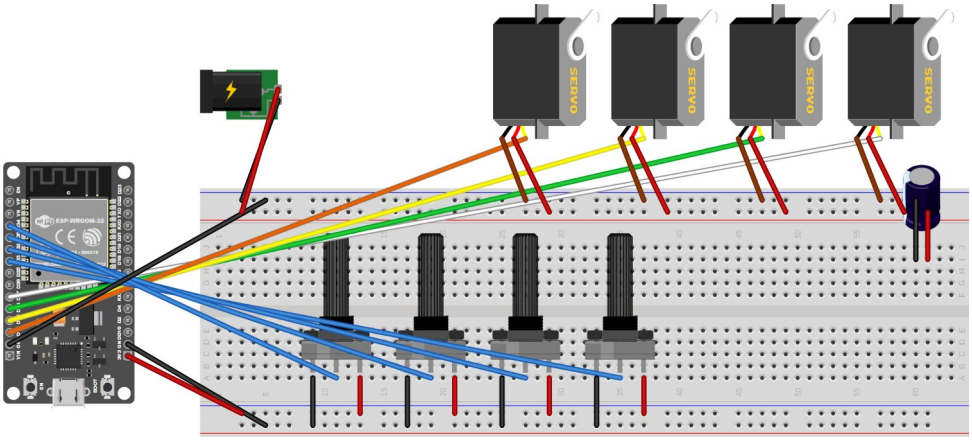


Figura 4 — Montagem do Encontro 5 (sketch 05). Verificar com multímetro o GND comum entre fonte e ESP32 antes de energizar.

Encontro 6 — Teach & repeat: linha de produção

Tempo	Atividade	Professor	Alunos	Evidência
10'	Como se programa um robô industrial	Vídeo curto de teach pendant; conceito de waypoint	—	—
20'	Sketch 06	Explica `struct`, `Preferences`, debounce, interpolação simultânea	Gravam	—
30'	Roteiro do ciclo (7 poses)	Cobra a pose de aproximação; provoca falhas mostrando um ciclo sem waypoint	Ensinam o braço	Ciclo pegar-e-soltar funcionando
25'	Desafios 6.1–6.4	Diferenciação: 6.1 para todos; 6.3/6.4 para os avançados; 6.5 se houver o painel MQTT do laboratório	Programam	Sketch + contador
15'	Apresentação	Cada grupo demonstra 3 ciclos e explica uma decisão de projeto	Apresentam	**Rubrica (§8)**

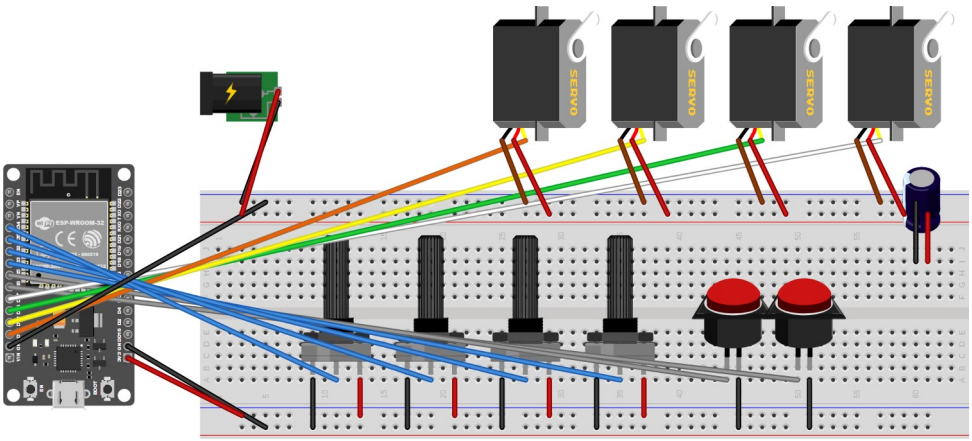


Figura 5 — Montagem do Encontro 6 (sketch 06): acrescenta os botões GRAVAR (GPIO 25) e PLAY (GPIO 26).

5. Fundamentação técnica para o professor

O sinal do servo

O servo analógico lê a largura do pulso (0,5–2,4 ms) e a compara com a posição do potenciômetro interno; o erro aciona o motor até zerar. O período de 20 ms (50 Hz) é uma convenção histórica de rádio-controle; servos aceitam de ~40 a ~200 Hz, mas a **largura** é o que importa. A biblioteca `Servo` do AVR usa o Timer1 e suporta 12 servos no Uno; a `ESP32Servo` usa o periférico LEDC e precisa de um timer para cada 2–4 canais — daí `allocateTimer()`.

Pulso fora da faixa: o servo tenta ir além do batente mecânico, o motor fica em stall (até 650 mA), esquenta e as engrenagens de nylon podem espanar. É a principal causa de morte de SG90 em sala. A calibração (sketch 03) existe para isso.

Corrente e alimentação

- Um SG90 movendo: 100–250 mA; 4 partindo juntos: pico > 1 A. O regulador do Uno (via USB) aguenta ~500 mA; o do ESP32, menos. **Fonte externa é obrigatória no braço.**
- **Brownout:** a queda momentânea de 5 V ao partir os servos derruba o 3,3 V do ESP32 → reset ("Brownout detector was triggered" no Serial). Capacitor de 470–1000 µF nos servos + movimento suave (1°/ciclo) resolvem.
- **GND comum:** sem ele, o sinal não tem referência; sintomas: tremor, servo "bêbado", não responde. Primeira coisa a verificar sempre.
- O sinal de 3,3 V do ESP32 é aceito pelo SG90 (limiar ~2 V). Não é preciso conversor de nível.

ADC do ESP32

O ADC de 12 bits do ESP32 é **não linear** e satura acima de ~3,1 V com atenuação de 11 dB: as pontas do pot "achatam". Para a aula isso é ótimo — dá para discutir erro de medida — e na prática é irrelevante porque o `map()` cobre a faixa toda. `analogSetAttenuation(ADC_11db)` é o padrão; GPIO 34–39 são só entrada; ADC2 (GPIO 0, 2, 4, 12–15, 25–27) **para de funcionar com Wi-Fi ligado** — por isso os pots estão no ADC1.

Interpolação e waypoints

`irPara()` avança todas as juntas 1° por passo até todas chegarem: é uma interpolação linear no espaço das juntas. Como as juntas têm cursos diferentes, a que tem mais a percorrer termina por último — o caminho da garra no espaço não é uma reta. Isso justifica os waypoints e é uma porta de entrada para cinemática (Desafio "para ir além").

Por que Uno primeiro e ESP32 depois?

O Uno tem menos armadilhas (5 V em tudo, ADC linear, biblioteca nativa, upload sem BOOT). Dominar o conceito numa plataforma simples e **migrar** é uma habilidade de engenharia em si. Se a escola só tiver ESP32, a sequência funciona com os sketches 04–06 desde o Encontro 1, adaptando 01–03 (mudanças: `ESP32Servo`, `0..4095`, pot no 3V3).

6. Erros comuns e como resolver

Sintoma	Causa provável	Solução
Servo treme, “ronca” ou não responde	GND comum ausente; alimentação fraca; fio laranja no pino errado	Multímetro: continuidade entre GND da fonte e da placa. Fonte ≥ 2 A. Conferir pino no código.
Servo não chega a 0° ou 180°	Limites do `attach()` conservadores (544–2400 padrão)	Usar os pulsos da Folha A no `attach()`.
Servo força e esquenta num extremo	Pulso além do batente; `ANG_MAX` alto; garra apertando	Reduzir limites; desligar imediatamente.
ESP32 reinicia sozinho (“Brownout”)	Servos puxando corrente do USB/3V3	Fonte externa + capacitor + movimento suave.
`Servo.h` não compila no ESP32	Biblioteca errada	Instalar e incluir `ESP32Servo.h`.
Porta COM não aparece	Driver CP210x/CH340 ausente; cabo só de carga	Instalar driver; trocar cabo por um de dados.
“Failed to connect to ESP32”	Auto-reset não funcionou	Segurar BOOT ao iniciar o upload; soltar ao ver “Connecting”.
`analogRead` sempre 4095 ou 0 no ESP32	Pot no 5 V (saturado) ou pino ADC2 com Wi-Fi; GPIO só-entrada usado como saída	Pot no 3V3; usar GPIO 32–39.
Pot invertido	Pinos das pontas trocados	Trocar os fios das pontas ou inverter o `map()`.
Braço bate na base / não alcança	Servo montado fora do centro	Desmontar o horn, centralizar (sketch 01 em `write(90)`), remontar.
Braço “tranca” ou tomba ao mover	Movimento brusco; base leve	Aumentar `PASSO_MS`; fixar a base com fita dupla-face ou peso.
Poses somem ao desligar	Não chamou `salvar()`; namespace do Preferences diferente	Conferir que `gravarPose()` chama `salvar()`.
Botão grava 3 poses num clique	Sem debounce; botão sem `INPUT_PULLUP`	Usar `apertou()`; conferir `pinMode`.
Garra não segura o objeto	`ANG_MAX` da garra baixo demais ou objeto largo/pesado	Aumentar 5° por vez; objeto ≤ 5 cm e leve.

7. Respostas e comentários dos desafios

Encontro 1

- **1.1** `write(0); delay(1000); write(90); delay(1000); write(180); delay(1000);` — observar que o servo vai "de uma vez"; ótimo gancho para a interpolação.
- **1.2** Dois `for` com delays diferentes (ex.: 3 ms na ida, 20 ms na volta).
- **1.3** `digitalWrite(13, HIGH)` antes do primeiro `for` e `LOW` antes do segundo.

Encontro 2

- **2.1** `if (abs(angulo - ultimo) > 2) { servo.write(angulo); ultimo = angulo; }` — a zona morta elimina o tremor causado pelo ruído do ADC.
- **2.2** Segundo objeto `Servo`, `attach(10)`, `analogRead(A1)`.
- **2.3** `if (alvo > atual) atual++; else if (alvo < atual) atual--; servo.write(atual); delay(15);` — é exatamente o núcleo dos sketches 05/06.
- **3.1** `t0 = millis(); write(180); while (…)` não dá para saber quando chegou (servo não tem feedback externo): discutir malha aberta vs. fechada. Medir com `delay` calibrado: ≈ 300–400 ms.
- **3.2** Diferenças típicas: 20–80 µs nos extremos (≈ 2–8°).

Encontro 3

- **4.1** Linhas a mudar: include, `Serial.begin(115200)`, `map(..., 0, 4095, ...)`, pinos, `setPeriodHertz(50)`, `allocateTimer`.
- **4.2** Média de 8 leituras reduz o ruído em ~√8 ≈ 2,8×; o movimento fica visivelmente mais estável. Custo: 8× o tempo de leitura (irrelevante aqui).

Encontro 5

- **5.1** Botão em `INPUT_PULLUP`; ao apertar, `irPara(poseDescanso)`.
- **5.2** 3°/ciclo: mais rápido, mais tranco, mais corrente. 0,5°/ciclo (float): muito suave, lento. Trade-off velocidade × suavidade × consumo — ideal para discussão.
- **5.3** LDR em divisor com 10 kΩ; quando a leitura cair abaixo de um limiar (objeto sombreia), fechar a garra. Introduz sensor + atuador em malha fechada.

Encontro 6

- **6.1** Tipicamente entre 3 e 6 ms/grau o objeto começa a escapar: inércia × atrito da garra. Relação direta com "takt time" em uma linha real.
- **6.2** `struct Pose { uint8_t ang[N]; uint16_t pausaMs; }` e usar `pose.pausaMs` no lugar de `PAUSA_POSE_MS`. Lembrar de ajustar `putBytes/getBytes` (o tamanho do struct muda).
- **6.3** Variável `ciclos++` ao final do `for` de poses; `Serial.printf`.
- **6.4** Antes de iniciar o ciclo: `while (digitalRead(SENSOR) == HIGH) delay(10);` — ou, melhor, checar no `loop` sem bloquear.
- **6.5** `PubSubClient` + Wi-Fi: `client.publish("sensores/braco", "{\"ciclos\":N}")`. A ponte MQTT→InfluxDB do laboratório grava e o Grafana mostra.

- **6.6** Uno: EEPROM de 1 KB (sobra), mas só 6 entradas analógicas, menos RAM e sem Wi-Fi; `EEPROM.update` tem vida útil de ~100 mil escritas — gravar a cada pose está OK, gravar em loop não.

8. Avaliação

Rubrica do projeto final (por grupo)

Critério	Iniciante (1)	Em desenvolvimento (2)	Proficiente (3)
Funcionamento	O braço move mas não completa o ciclo	Completa o ciclo com ajuda ou falha ocasional	Completa ≥ 3 ciclos seguidos sem intervenção
Calibração e segurança	Folhas incompletas; servo forçando	Folhas preenchidas; limites parcialmente ajustados	Folhas A e B completas; limites justificados; nenhum servo forçando
Compreensão do código	Reproduz sem explicar	Explica o que cada bloco faz	Explica **por que** (pulso, interpolação, waypoints, persistência) e modifica com propósito
Depuração	Depende do professor	Usa o Serial para localizar o problema	Diagnostica pela tabela sintoma→causa e resolve autonomamente
Colaboração e comunicação	Um aluno faz tudo	Papéis definidos	Todos explicam uma parte; apresentação clara com demonstração

Avaliação formativa ao longo do curso

- Tickets de saída (1 pergunta por encontro, listadas nos planos).
- Folhas de calibração A e B — evidência de rigor de medida.
- Sketches dos desafios — evidência de transferência. Sugestão: pedir um comentário no topo de cada sketch explicando a modificação.
- Autoavaliação simples: “O que eu sei fazer sozinho / com ajuda / ainda não sei” para os 8 objetivos do §1.

9. Diferenciação e inclusão

- **Grupos heterogêneos** com papéis rotativos: montador, programador, “engenheiro de testes” (preenche folhas e checklist).
- **Alunos avançados:** Desafios 6.3–6.5, servidor web no ESP32, controle por Python/Node-RED, cinemática.
- **Alunos com dificuldade:** fornecer os sketches com os pontos de alteração marcados por `// <<< MUDE AQUI`; permitir que a calibração seja feita em dupla com um colega.
- **Baixa visão:** knobs grandes nos pots; Monitor Serial com fonte ampliada; etiquetas em alto contraste nos fios (marrom/vermelho/laranja).

- **Sem impressora 3D:** o percurso até o Encontro 3 é completo por si; o braço pode ser substituído por 2 servos em uma “garra” de papelão ou por um pan-tilt de acrílico.

10. Segurança no laboratório

- Fontes 5 V com plugue certo e cabos íntegros; nada ligado à rede além das fontes.
- Braço em modo automático só com área livre; dedos longe da garra.
- Servos travados esquentam: desligar ao sinal de ronco.
- Estilete e ferro de solda (se usado) apenas com supervisão.
- Impressão 3D fora do horário de aula (fumos do PLA são baixos, mas a impressora aquece a 200 °C).

11. Referências

- Servo Mini Arm — LucaDilo — https://makerworld.com/models/1290483 (peças, perfis de impressão, Assembly Sheet).
- Código de referência do autor — https://github.com/LucaDiLorenzo98/mini_servo_arm_code
- Tower Pro SG90 — datasheet (dimensões, pulso 500–2400 µs, 4,8–6 V).
- Arduino — referência da biblioteca Servo: https://www.arduino.cc/reference/en/libraries/servo/
- ESP32Servo — https://github.com/madhephaestus/ESP32Servo
- ESP32 Arduino Core — ADC e Preferences: https://docs.espressif.com/projects/arduino-esp32/
- Sketches deste curso: pasta `aula-servos/codigo/` (01–07).

Anexo A — Sketches de referência

Os sketches completos estão no Guia do Aluno e na pasta de código. Reproduzimos aqui apenas o sketch final, com os pontos de intervenção didática comentados.

06_gravar_reproduzir_esp32.ino

```
/*
  06 - Linha de producao: gravar poses e reproduzir automaticamente
  Placa: ESP32 Dev Module + ESP32Servo + Preferences (memoria flash)

  Ideia (igual a um robo industrial em modo "teach"):
    1. MODO MANUAL: voce posiciona o braco com os 4 potenciometros.
    2. Aperta o botao GRAVAR: a pose atual (4 angulos) entra na lista.
    3. Repete para cada etapa do ciclo (pegar, levantar, girar, soltar, voltar...).
    4. Aperta PLAY: o braco repete a sequencia sozinho, em loop, como numa esteira.
    5. As poses ficam salvas na flash: sobrevivem a desligar o ESP32.

  Botoes (com INPUT_PULLUP: um lado no GPIO, outro no GND):
    GRAVAR -> GPIO 25     PLAY/PARAR -> GPIO 26     APAGAR TUDO -> segurar GRAVAR 3 s

  Comandos pelo Monitor Serial (115200) tambem funcionam:
    g = gravar pose   p = play/parar   l = listar   x = apagar tudo

  Servos e pots: mesmas ligacoes do sketch 05.
*/
#include <ESP32Servo.h>
#include <Preferences.h>

const int N = 4;
const int PINO_SERVO[N] = {13, 12, 14, 27};
const int PINO_POT[N]   = {34, 35, 32, 33};
const int ANG_MIN[N]    = {  0,  20,  10,  40 };
const int ANG_MAX[N]    = {180, 160, 170, 120 };
const int PINO_BTN_GRAVAR = 25;
const int PINO_BTN_PLAY   = 26;

const int MAX_POSES = 20;
struct Pose { uint8_t ang[N]; };
Pose poses[MAX_POSES];
int numPoses = 0;

// Parametros do ciclo automatico (mexa aqui para mudar o "ritmo da linha")
const int PASSO_MS       = 15;    // ms por grau ao interpolar: menor = mais rapido
const int PAUSA_POSE_MS  = 600;   // tempo parado em cada pose (ex.: fechar a garra)

Servo servos[N];
int anguloAtual[N];
bool modoPlay = false;
Preferences prefs;

// ---------- persistencia ----------
void salvar() {
  prefs.begin("braco", false);
  prefs.putInt("n", numPoses);
  prefs.putBytes("poses", poses, sizeof(Pose) * numPoses);
  prefs.end();
}
void carregar() {
  prefs.begin("braco", true);
  numPoses = prefs.getInt("n", 0);
  if (numPoses > MAX_POSES) numPoses = 0;
  if (numPoses > 0) prefs.getBytes("poses", poses, sizeof(Pose) * numPoses);
  prefs.end();
}

// ---------- movimento ----------
void irPara(const uint8_t alvo[N]) {          // interpola todas as juntas juntas
  bool chegou = false;
  while (!chegou) {
```

```
    chegou = true;
    for (int i = 0; i < N; i++) {
      if (anguloAtual[i] < alvo[i])      { anguloAtual[i]++; chegou = false; }
      else if (anguloAtual[i] > alvo[i]) { anguloAtual[i]--; chegou = false; }
      servos[i].write(anguloAtual[i]);
    }
    delay(PASSO_MS);
  }
}

void lerPots() {
  for (int i = 0; i < N; i++) {
    int alvo = map(analogRead(PINO_POT[i]), 0, 4095, ANG_MIN[i], ANG_MAX[i]);
    if (alvo > anguloAtual[i]) anguloAtual[i]++;
    else if (alvo < anguloAtual[i]) anguloAtual[i]--;
    servos[i].write(anguloAtual[i]);
  }
}

// ---------- acoes ----------
void gravarPose() {
  if (numPoses >= MAX_POSES) { Serial.println("Lista cheia!"); return; }
  for (int i = 0; i < N; i++) poses[numPoses].ang[i] = anguloAtual[i];
  numPoses++;
  salvar();
  Serial.printf("Pose %d gravada: ", numPoses);
  for (int i = 0; i < N; i++) Serial.printf("%d ", anguloAtual[i]);
  Serial.println();
}
void listar() {
  Serial.printf("%d pose(s):\n", numPoses);
  for (int p = 0; p < numPoses; p++) {
    Serial.printf("  #%2d  base=%3d ombro=%3d cotovelo=%3d garra=%3d\n", p + 1,
                  poses[p].ang[0], poses[p].ang[1], poses[p].ang[2], poses[p].ang[3]);
  }
}
void apagarTudo() { numPoses = 0; salvar(); Serial.println("Poses apagadas."); }

// ---------- botoes com debounce simples ----------
bool apertou(int pino) {
  static unsigned long ultimo[40] = {0};
  if (digitalRead(pino) == LOW && millis() - ultimo[pino] > 300) {
    ultimo[pino] = millis();
    return true;
  }
  return false;
}

void setup() {
  Serial.begin(115200);
  pinMode(PINO_BTN_GRAVAR, INPUT_PULLUP);
  pinMode(PINO_BTN_PLAY,   INPUT_PULLUP);
  analogReadResolution(12);
  analogSetAttenuation(ADC_11db);
  ESP32PWM::allocateTimer(0);
  ESP32PWM::allocateTimer(1);
  for (int i = 0; i < N; i++) {
    servos[i].setPeriodHertz(50);
    servos[i].attach(PINO_SERVO[i], 500, 2400);
    anguloAtual[i] = (ANG_MIN[i] + ANG_MAX[i]) / 2;
    servos[i].write(anguloAtual[i]);
  }
  carregar();
  Serial.println("g=gravar  p=play/parar  l=listar  x=apagar");
  listar();
}

void loop() {
  // --- comandos serial ---
  if (Serial.available()) {
    char c = Serial.read();
    if (c == 'g') gravarPose();
    if (c == 'p') modoPlay = !modoPlay;
    if (c == 'l') listar();
```

```
    if (c == 'x') apagarTudo();
  }
  // --- botoes ---
  if (apertou(PINO_BTN_PLAY)) { modoPlay = !modoPlay; Serial.println(modoPlay ? "PLAY" : "PARADO"); }
  if (digitalRead(PINO_BTN_GRAVAR) == LOW) {
    unsigned long t0 = millis();
    while (digitalRead(PINO_BTN_GRAVAR) == LOW) {
      if (millis() - t0 > 3000) { apagarTudo(); while (digitalRead(PINO_BTN_GRAVAR) == LOW); return; }
    }
    gravarPose();
  }

  if (modoPlay && numPoses > 0) {
    for (int p = 0; p < numPoses && modoPlay; p++) {
      irPara(poses[p].ang);
      delay(PAUSA_POSE_MS);
      if (apertou(PINO_BTN_PLAY) || (Serial.available() && Serial.read() == 'p')) {
        modoPlay = false; Serial.println("PARADO");
      }
    }
  } else {
    lerPots();          // modo manual: segue os potenciometros
    delay(10);
  }
}
```

Anexo B — Folhas de calibração (cópias para distribuir)

Folha A — pulsos do servo (sketch 03)

Servo nº	Pulso 0° (µs)	Pulso 180° (µs)	Centro (µs)	Usado em (junta)	Obs.
1					
2					
3					
4					
5					
6					

Folha B — limites mecânicos (sketch 05)

Junta	Servo nº	Encosta (mín.)	ANG_MIN	Encosta (máx.)	ANG_MAX
Base					
Ombro					
Cotovelo					
Garra					

Tabela de correntes (Encontro 5)

Situação	Corrente medida (mA)	Tensão na fonte (V)	ESP32 reiniciou?
1 servo parado			
1 servo movendo			
4 servos movendo			
Garra forçando (1 s, no máximo!)			